

Matteo Calvanico

☎ +39-347-385-2782 ✉ matteo.calvanico03@gmail.com [in Matteo Calvanico](#) [🌐 MatteoCalvanico](#) [📁 Portfolio](#)

COMPETENZE

- **Linguaggi:** Java, C# (.NET 8.0), C++, C89, Python, Kotlin, Javascript, Typescript, SQL, GDScript, LaTeX.
- **Framework & Librerie:** Vue, React, Jest, Zod, Mongoose, Serverless Application Model (SAM), Entity Framework, .NET, Bootstrap, TensorFlow, Scikit Learn, PyTorch, CUDA, OpenCV, Pandas, Numpy, Gradle.
- **Tools:** Visual Studio, VS Code, Andorid Studio, Arduino, Google Cloud Console, AWS, Git, GitHub, Godot, Docker, Jenkins, Dockerhub.

FORMAZIONE

- **"Alma Mater Studiorum" - Università di Bologna** Cesena (FC), Italia
Laurea triennale in Tecnologie dei Sistemi Informatici 2022 – 2025
 - **Competenze:** Object-Oriented Programming (Java, C#), Programmazione di sistemi mobili (Kotlin, Android Studio, Room, Gradle, Retrofit), Basi di dati (SQL, MySQL), Ingegneria dei sistemi web (JavaScript, TypeScript, CSS, SCSS/SASS, NodeJS, Vue.js, Bootstrap), Basi di Reti Neurali, Machine Learning, Elaborazione di immagini (OpenCV, CUDA, Python, PyTorch, TensorFlow, Scikit Learn, Pandas), Sistemi Embedded e Internet of Things (Arduino, ESP32, Raspberry Pi, MQTT), Piattaforme di sviluppo per automazione e virtualizzazione (Docker, Vagrant, Kubernetes, Active Directory), Sistemi di rete, Sviluppo in Asp.NET
- **I.T.T. "Blaise Pascal"** Cesena (FC), Italia
Diploma da perito informatico Settembre 2017 – Giugno 2022
 - **Descrizione:** Ho completato la scuola superiore con una specializzazione in Informatica e Sistemi di Telecomunicazione, studiando le basi della programmazione, dei sistemi di rete, dell'elettronica e molto altro.

ESPERIENZE LAVORATIVE

- **DMA - CX Company for B2B** Cesena (FC), Italia
Intern Software Engineer Aprile 2025 – Agosto 2025
 - **Tecnologie usate:** AWS (Cloudformation, Lambda, API Gateway, SQS, Timestream, IoT Core, EC2), React, Grafana, Docker, GitHub Actions, MongoDB Atlas
 - **Descrizione:** La collaborazione con DMA ha portato allo sviluppo di una *Dashboard*, dove l'operatore del servizio clienti di un'azienda, che si affida a DMA per le sue soluzioni software, può monitorare lo stato attuale e passato dei diversi dispositivi usati dai vari consumatori; in modo da poter fornire un supporto mirato e funzionale in caso di guasti o problemi di qualsiasi tipo. La creazione della dashboard non è stata immediata, prima di iniziare la progettazione e lo sviluppo del prodotto è stato necessario apprendere nuove conoscenze e consolidarne altre su varie tecnologie. Il processo è stato a step, con lo sviluppo di progetti più piccoli e di difficoltà crescenti, che raccogliessero le varie conoscenze necessarie per lo sviluppo della dashboard.
- **Maggioli S.P.A.** Santarcangelo di Romagna (FC), Italia
Intern Software Engineer Novembre 2024 – Marzo 2025
 - **Tecnologie usate:** Angular, Java, Spring, Postman, Docker, MongoDB, GitLab
 - **Descrizione:** Azienda italiana specializzata in sviluppo di applicazioni per la pubblica amministrazione. Lavorando come programmatore Full-Stack ho imparato a muovermi in una code base già consolidata ed a lavorare in un grande team, imparando come affrontare i problemi quotidiani che si possono presentare in ogni momento in una azienda grande e dinamica.

PROGETTI

- **ECC Dashboard:** Applicativo con componenti cloud per il supporto clienti.
 - **AWS:** l'applicativo, nella sua parte back-end, utilizza diverse tecnologie della suite AWS per funzionare. Tra queste: **IoT Core** per ottenere i dati dai dispositivi di campo, **Timestream** per salvare i dati in arrivo, **Lambda** per effettuare operazioni sulle fonte dati e altre ancora.

- **Grafana e React:** il front-end è diviso in due parti. La dashboard utilizza Grafana per mostrare i dati inviati dai dispositivi, con la possibilità di filtrare e visualizzare quelli passati. Il portale con le informazioni relative a dispositivi, utenti e allarmi sono mostrati tramite un applicativo sviluppato in React.
 - **Deploy automatizzato:** l'intera infrastruttura cloud più l'applicativo front-end sono rilasciati tramite un processo automatizzato realizzato tramite GitHub Actions.
 - **Tesi:** la progettazione e sviluppo dell'intero applicativo sono stati documentati nella mia tesi di laurea.
- **Game Vault:** Android app per il tracking della propria collezione di giochi, con la possibilità di rating e ricerca.
 - **Play Store:** l'applicazione è stata testata da 20 persone e approvata da Google, ora è disponibile per il download direttamente dal Google Play Store
 - **API:** la ricerca viene fatta tramite api pubbliche, che forniscono tutte le informazioni necessarie sul gioco per poi essere mostrate all'utente; i file Json ritornati venivano gestiti tramite Moshi e Retrofit.
 - **DB:** per salvare le informazioni in maniera persistente si è utilizzato Room.
- **GetEat:** Web app per la gestione di un ristorante, con la possibilità di acquistare prodotti per gli utenti e di gestirli da parte dell'admin.
 - **Multi-tecnologia:** la web app è stata creata tramite il framework Vue.js, utilizzando TypeScript e Scss con l'utilizzo di Express.js per gestire le chiamate al DB relazionale creato tramite SQL.
 - **Modularità:** l'applicazione è stata divisa prima in Front-end/Back-end per poi passare ad una suddivisione ancora maggiore in file modulari all'interno di ciascuna parte.
 - **Autenticazione:** l'applicazione prevede una parte di registrazione e login con controllo dei vari permessi, andando così a nascondere agli utenti base la parte dell'amministratore
 - **Accessibilità:** l'obiettivo non era solo quello di avere una applicazione funzionante ma di avere anche una web app responsive e soprattutto accessibile.
- **Virtual Casinò:** Applicativo Desktop in Java che simuli alcuni giochi di un casinò, come Blackjack, Roulette e i dadi.
 - **MVC:** per rendere il progetto meno complesso e favorire una facile modifica e testabilità si è deciso di usare il Pattern MVC, andando a dividere la parte di presentazione da quella dei dati garantendo la comunicazione tra i due tramite il controller
 - **Collaborazione:** il progetto è sviluppato insieme ad altri due colleghi, questo ci ha permesso di avere un primo assaggio sul lavoro in team, portando a suddividere il lavoro in maniera equa, sentirci e discutere sul da farsi in maniera regolare tramite call virtuali e aiutarci in caso di difficoltà.
 - **Test-Driven Development:** una parte cruciale del progetto era quella di effettuare test periodici e automatizzati per far sì che ogni parte funzioni correttamente, questo è stato possibile grazie a JUnit.
 - **Gradle:** per automatizzare la compilazione e l'esecuzione di test abbiamo utilizzato Gradle, che ci è stato molto utile anche per la gestione delle dipendenze.
- **Mako:** Web app in .NET per la gestione di un porto navale con relativi turni di scarico/carico e lavoratori associati.
 - **Design:** una buona parte del tempo impiegato per il progetto è stato speso nella progettazione dell'applicativo, con enorme focus sulla UX/UI.
 - **ASP.NET Core:** la web app è stata sviluppata utilizzando ASP.NET Core, utilizzando il design pattern MVC, e ovviamente C# come linguaggio principale.
 - **Entity Framework:** per l'interfacciamento con il database si è deciso di usare questo ORM, rendendo la creazione e gestione delle query/database facile e veloce grazie a LINQ.
- **MultiClass Classification:** Rete neurale per la creazione e test di un modello di machine learning per la classificazione multiclasse di diversi tipi di frutta. La rete è una classica CNN con una funzione di attivazione non lineare (ReLU) con anche un ottimizzatore di tipo SGD.
 - **CONDA:** Anaconda è stato fondamentale per creare un environment con tutto il necessario per lo sviluppo e la prova della rete.
 - **Multi-libreria:** Per creare, addestrare e testare la rete è stato necessario l'utilizzo di diverse librerie come Pytorch o OpenCV che ha permesso di modificare le immagini del dataset per aggiustare certi aspetti.
 - **Kaggle:** per trovare un dataset che fosse perfetto per le mie esigenze ho effettuato diverse ricerche per poi finire su Kaggle, che mi ha permesso anche di utilizzare le sue API per il download delle immagini.
- **DoomClone:** Videogioco sviluppato in C++ simile ai vecchi classici come Doom o Wolfenstein 3D.
 - **Basi di computer grafica:** per creare un effetto 3D tramite una mappa 2D mi sono cimentato nella tecnica del Raycasting, dove si utilizzano diverse formule matematiche per mostrare a schermo il contenuto di un vettore (che contiene la mappa), questo grazie a diversi "raggi" che quando colpiscono un punto dove dovrebbe stare un muro o un nemico ne permettono di ottenere la distanza e successivamente disegnarlo; questo viene fatto per ogni linea verticale dello schermo.

- **SDL2:** per mostrare a schermo il render e gestire i vari eventi è stato necessario usare la libreria SDL2.
- **Event Handle:** per gestire diverse casistiche come l'eliminazione dei nemici, il movimento o l'apertura della porta è stato necessario un handle degli eventi nel loop principale del gioco, sempre tramite SDL2.
- **ProtoCRPG:** Prototipo di un CRPG sviluppato in Godot 4.3 come punto di partenza per giochi simili.
 - **Project based learning:** lo scopo del progetto è quello di imparare non solo la tecnologia in se (Godot Engine e GDScript) ma anche come si lavora ad un vero gioco, con tutti i problemi del caso.
 - **Passione:** l'obiettivo finale è quello di avere una base solida per iniziare a lavorare, in futuro, su un mio personale gioco di ruolo isometrico, seguendo le orme di vecchi classici come Fallout e Baldur's Gate
 - **Perchè Godot:** la scelta di questo Game Engine non è casuale, la decisione di scegliere un tool open source è dovuta alla possibilità di essere libero di sperimentare e studiare anche come funzionano queste tecnologie dietro le quinte.